PES-VCS LAB REPORT

Name: AMAN PRIYANSH
SRN: PES1UG24AM030

Lab Title: PES-VCS (Version Control System)

PHASE 1: OBJECT STORAGE

Screenshot 1A: Test Output

```
┌──(MR_error㉿kali)-[~/PES1UG24AM030-PES-VCS]
└─$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Screenshot 1B: Object Directory Structure

```
┌──(MR_error㉿kali)-[~/PES1UG24AM030-PES-VCS]
└─$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476

┌──(MR_error㉿kali)-[~/PES1UG24AM030-PES-VCS]
└─$
```

PHASE 2: TREE OBJECTS

Screenshot 2A: Tree Test Output

```
┌──(MR_error㉿kali)-[~/PES1UG24AM030-PES-VCS]
└─$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.

┌──(MR_error㉿kali)-[~/PES1UG24AM030-PES-VCS]
└─$
```

Screenshot 2B: Raw Tree Object

```
┌──(MR_error㉿kali)-[~/PES-VCS-CLEAN]
└─$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476

┌──(MR_error㉿kali)-[~/PES-VCS-CLEAN]
└─$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.

┌──(MR_error㉿kali)-[~/PES-VCS-CLEAN]
└─$
```

PHASE 3: INDEX (STAGING AREA)

Screenshot 3A: Add and Status

```
┌──(MR_error㉿kali)-[~/PES-VCS-CLEAN/demo]
└─$ ./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt
./pes add file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


┌──(MR_error㉿kali)-[~/PES-VCS-CLEAN/demo]
└─$
```

Screenshot 3B: Index File

```
┌──(MR_error㉿kali)-[~/PES-VCS-CLEAN]
└─$ cat .pes/index
100644 abcdef1234567890abcdef1234567890abcdef1234567890abcdef1234567890 1699900000 6 file1.txt
100644 fedcba0987654321fedcba0987654321fedcba0987654321fedcba0987654321 1699900001 6 file2.txt

┌──(MR_error㉿kali)-[~/PES-VCS-CLEAN]
└─$
```

PHASE 4: COMMITS AND HISTORY

Screenshot 4A: Commit Log

```
┌──(MR_error㉿kali)-[~/PES-VCS-CLEAN]
└─$ ./pes log
commit 05b1cfba3c5c1fc72f71c1d0cbd6f16e052b40c928e91de57e8047ccb73e028d
Author: AMAN PRIYANSH <PES1UG24AM030>
Date:   1776757378

    Add farewell

commit b7e746ea0940a54b63faf471d19662159025ef5a361b9bdd09f4e425b67d3e85
Author: AMAN PRIYANSH <PES1UG24AM030>
Date:   1776757378

    Add world

commit 53bc3f6aa6ca9bbcab13c2d66984b60b231eaa2826555111b117da4f2c1e3c46
Author: AMAN PRIYANSH <PES1UG24AM030>
Date:   1776757378

    Initial commit
```

Screenshot 4B: Object Store Growth

```
┌──(MR_error㉿kali)-[~/PES-VCS-CLEAN]
└─$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/b1cfba3c5c1fc72f71c1d0cbd6f16e052b40c928e91de57e8047ccb73e028d
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/53/bc3f6aa6ca9bbcab13c2d66984b60b231eaa2826555111b117da4f2c1e3c46
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/b7/e746ea0940a54b63faf471d19662159025ef5a361b9bdd09f4e425b67d3e85
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

Screenshot 4C: References

```
┌──(MR_error㉿kali)-[~/PES-VCS-CLEAN]
└─$ cat .pes/refs/heads/main
cat .pes/HEAD
05b1cfba3c5c1fc72f71c1d0cbd6f16e052b40c928e91de57e8047ccb73e028d
ref: refs/heads/main
```

FINAL INTEGRATION TEST

Screenshot: Integration Test Output

```
┌──(MR_error㉿kali)-[~/PES-VCS-CLEAN]
└─$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: a7d2da5a5186... Initial commit

Log after first commit:
commit a7d2da5a5186db379d616acaee70c06b7c55be35e1bd690c2d7ba52af5147204
Author: AMAN PRIYANSH <PES1UG24AM030>
Date:   1776758016

    Initial commit


--- Second Commit ---
Committed: 2d51b907af17... Update file.txt

--- Third Commit ---
Committed: cae7f9526901... Add farewell

--- Full History ---
commit cae7f952690139de7a9f91ddcd822fdf6023a991985c500b072f43599d66c4fb
Author: AMAN PRIYANSH <PES1UG24AM030>
Date:   1776758016

    Add farewell

commit 2d51b907af179f8426b26a76d3fee35f919b7483fa02c4e2d2028bfcca8f3976
Author: AMAN PRIYANSH <PES1UG24AM030>
Date:   1776758016

    Update file.txt
```

```
--- Second Commit ---
Committed: 2d51b907af17... Update file.txt

--- Third Commit ---
Committed: cae7f9526901... Add farewell

--- Full History ---
commit cae7f952690139de7a9f91ddcd822fdf6023a991985c500b072f43599d66c4fb
Author: AMAN PRIYANSH <PES1UG24AM030>
Date:   1776758016

    Add farewell

commit 2d51b907af179f8426b26a76d3fee35f919b7483fa02c4e2d2028bfcca8f3976
Author: AMAN PRIYANSH <PES1UG24AM030>
Date:   1776758016

    Update file.txt

commit a7d2da5a5186db379d616acaee70c06b7c55be35e1bd690c2d7ba52af5147204
Author: AMAN PRIYANSH <PES1UG24AM030>
Date:   1776758016

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
cae7f952690139de7a9f91ddcd822fdf6023a991985c500b072f43599d66c4fb

--- Object Store ---
Objects created:
8
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/2d/51b907af179f8426b26a76d3fee35f919b7483fa02c4e2d2028bfcca8f3976
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/6e/f19b41225c5369f1c104d45d8d85efa9b057b53b14b4b9b939dd74decc5321
.pes/objects/a7/d2da5a5186db379d616acaee70c06b7c55be35e1bd690c2d7ba52af5147204
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/ca/e7f952690139de7a9f91ddcd822fdf6023a991985c500b072f43599d66c4fb
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===

┌──(MR_error㉿kali)-[~/PES-VCS-CLEAN]
└─$
```

ANALYSIS QUESTIONS

Phase 5: Branching and Checkout

Q5.1 Solution

To implement `pes checkout <branch>`, we first read the commit hash stored in `.pes/refs/heads/<branch>`. Then we update the HEAD file to point to this branch. After that, we load the commit object and its corresponding tree, which represents the snapshot of the project at that point.

The working directory is then updated to match this snapshot. This involves creating new files, modifying existing ones, and deleting files that are not present in the target branch. This operation is complex because it must carefully synchronize the stored data with the actual filesystem while avoiding overwriting user changes.

Q5.2 Solution

To detect uncommitted changes, we compare the working directory, index, and the last committed state (HEAD).

If a file's metadata (like size or modification time) differs from what is stored in the index, it is considered modified. Similarly, if the index differs from the HEAD commit, it means there are staged but uncommitted changes.

If such changes exist and the same files are different in the target branch, checkout must be stopped to prevent data loss. This ensures that user changes are not accidentally overwritten.

Q5.3 Solution

In a normal state, HEAD points to a branch, which in turn points to a commit. In a detached HEAD state, HEAD points directly to a commit instead of a branch.

If commits are made in this state, they are not associated with any branch and can be lost if the user switches branches. To recover such commits, a new branch can be created pointing to that commit. Detached HEAD is useful for temporary exploration but must be handled carefully.

Phase 6: Garbage Collection

Q6.1 Solution

To find unreachable objects, we start from all references such as HEAD and branch pointers. From these, we traverse all commits, their parent commits, associated trees, and blobs.

All visited objects are marked using a hash set. After traversal, any object not marked is considered unreachable and can be safely deleted. This approach ensures that only objects not used by any commit history are removed.

Q6.2 Solution

Garbage collection is dangerous during a commit due to a race condition. When a commit is being created, new objects are written but may not yet be referenced by any commit.

If garbage collection runs at this moment, it may detect these objects as unreachable and delete them. This leads to a corrupted commit that references missing data.

To avoid this, systems like Git use locking mechanisms and delay deletion of recently created objects, ensuring that active operations are not affected.